

SecureDT-6G: A Privacy-Preserving Framework for 6G Digital Twins

Executive Summary

Connected vehicles generate rich telemetry (speed, location, etc.) that powers Digital Twins for predictive maintenance and safety. However, this telemetry is highly sensitive (e.g. driver behavior, routes) and must remain private even when processed by cloud/MEC providers. Our **SecureDT-6G** framework addresses this gap by performing **encrypted Digital Twin inference**: vehicle data is CKKS-homomorphically encrypted on-board (TenSEAL library on the vehicle) and sent over 6G/MEC to a cloud-hosted inference engine (FastAPI + Redis). The cloud runs a simple **logistic regression** anomaly detector directly on ciphertext using polynomial approximations of nonlinearities, then returns an encrypted score that the vehicle decrypts. This “encrypt-all-the-way” design provides **IND-CPA privacy guarantees**: the cloud never sees raw telemetry ¹. Conceptually, this means the Digital Twin can operate on your encrypted speed/brake data without ever decrypting it.

Digital Twins are rapidly being adopted in automotive and smart-city applications. Industry analyses project the global DT market rising from \$21B in 2024 to ~\$29B by 2025, with automotive a leading sector ². In 2025, leaders from IBM, NVIDIA, BMW and others concur that vehicle digitization and AI integration are accelerating, making DTs “central” to how cars are built, tested, and maintained ³. SecureDT-6G advances this trend by adding **privacy**: recent studies confirm that homomorphic encryption can hide sensor data even in a live DT simulation ⁴. In one demonstration, an “untrusted” cloud node performed correct DT operations on ciphertext and never saw plaintext values ⁴. Our novelty is integrating CKKS-based HE with 6G/MEC DTs and measuring the feasibility trade-offs in a realistic vehicular setting. In sum, SecureDT-6G offers strong privacy (semantic security under RLWE ¹) with a practical prototype roadmap.

Keywords: Homomorphic Encryption, CKKS, Digital Twins, 6G, Privacy-Preserving Machine Learning, TenSEAL, Edge Computing, Federated Learning

1. Introduction and Background

Next-generation (6G) vehicular networks will rely on Digital Twins – real-time virtual replicas – for safety, efficiency, and predictive analytics ⁵ ³. A DT for a car continually mirrors its state (speed, acceleration, location, sensor readings) to enable anomaly detection, routing, maintenance, and more. However, raw vehicle telemetry is privacy-critical: eavesdropping or insider access could reveal personal travel patterns, driving styles, or sensitive environmental data. Standard transport security (TLS) protects data-in-transit, but once the data arrives at the cloud/MEC, it must be decrypted for analysis. This “last-mile” of trust is vulnerable: an honest-but-curious operator could misuse or leak the information.

Threat Model: We assume an honest-but-curious infrastructure provider (cloud/MEC operator) and no collusion with external parties. The provider follows protocols correctly (providing DT services) but may inspect any data at rest or in memory. We seek **data confidentiality**: the cloud should learn nothing about vehicle telemetry beyond the final prediction. Our system does *not* address Byzantine faults or

malicious clients. The security guarantee will be semantic (IND-CPA) encryption of all sensitive inputs, meaning ciphertexts leak no information about plaintexts ¹.

Problem Statement: How can we perform DT inference on highly sensitive vehicle data without exposing that data, even in a powerful 6G/MEC environment? Conventional DT architectures simply send plaintext telemetry to a cloud AI model, which violates privacy. SecureDT-6G's core idea is to replace plaintext pipelines with a **homomorphic encryption (HE) pipeline**: telemetry is encrypted on the vehicle (CKKS scheme ⁶) and the cloud runs the DT model on ciphertext. The vehicle then decrypts the output. Thus the cloud "sees" only random-looking ciphertext, preserving full privacy ⁴ ¹.

Contributions:

- **Privacy-Preserving DT:** We introduce a novel DT framework where *all* telemetry is CKKS-encrypted before leaving the vehicle, and inference (anomaly scoring) is done on ciphertext. This ensures a zero-trust posture toward the 6G MEC operator ⁴ ¹.
- **Hybrid System Design:** We detail an end-to-end architecture combining simulated vehicle data (CARLA) ⁷, encryption (TenSEAL & Microsoft SEAL), lightweight ML (logistic regression), and network emulation (tc netem on 6G-like links). We present module interfaces, APIs (FastAPI endpoints), and sequence/ER diagrams.
- **Feasibility Evaluation:** We perform a preliminary performance study, estimating latency, bandwidth, and resource costs. For example, CKKS ciphertexts are large (≈ 500 KB each ⁸), so 10 vehicles sending 10 Hz telemetry each would use ≈ 50 MB/s. At scale (1000 vehicles), this is multi-GB/s, which is **not feasible**. We quantify such limits and tune parameters (security vs. precision) accordingly.
- **Prototype Plan:** We outline a practical implementation plan with milestones, needed tools (Docker, Kubernetes, CI/CD), and team roles. We also enumerate ethical/regulatory considerations (e.g. GDPR) and potential industry applications (auto OEMs, smart cities).
- **Literature Context:** We survey recent work (2020–2026) on privacy-preserving ML, homomorphic logistics, and DTs in 6G/IoV. We highlight gaps in prior DT security work (which largely assumes trusted MEC) and show how SecureDT-6G uniquely fills those gaps.

In summary, SecureDT-6G is an ambitious but achievable research prototype. It combines state-of-the-art cryptography (CKKS ⁶), ML (LR with polynomial sigmoid), and networking (6G MEC). By carefully choosing simple models and evaluating network constraints, we can demonstrate end-to-end encrypted DT inference. We acknowledge that scalability is limited by HE costs and 6G rollout (simulated), but as a proof of concept it has high novelty, industry relevance, and publication potential.

2. System Architecture

The SecureDT-6G system has three main modules (Figure 1):

1. **Vehicle Data & Encryption (Client Side)** – The vehicle (or simulator) collects raw telemetry, extracts features, and encrypts them with CKKS.
2. **Encrypted Digital Twin Service (Cloud/MEC Side)** – A server hosts the Digital Twin inference service (FastAPI), which accepts ciphertext vectors, runs HE-based ML inference (logistic regression with polynomial activations), and returns encrypted scores.
3. **6G Network Emulation & Infrastructure** – We simulate 6G link characteristics (low-latency, high-bandwidth MEC) using Linux `tc netem`, and use Docker/Kubernetes to deploy services in containers near the edge.

We now describe each module and the data flow between them.

2.1 Vehicle-side Module: Data Generation & Encryption

- **Data Source:** In practice, a vehicle's CAN bus or sensors provide telemetry: speed, acceleration, brake pressure, steering angle, GPS, etc. In our prototype, we use the open-source **CARLA Simulator** ⁷ to generate realistic vehicle data (speed profiles, control inputs) in a virtual urban environment. CARLA is widely used for autonomous driving research and can output high-fidelity telemetry streams.
- **Feature Extraction:** From raw data, a small feature vector is formed each time step (e.g. [speed, acceleration, steering]). These values are floating-point real numbers, suitable for CKKS.
- **CKKS Encryption (TenSEAL):** We use the TenSEAL library (built on Microsoft SEAL) to perform CKKS encryption of the feature vector ⁹. TenSEAL provides a Python API (with C++ backend) to create a `TSCCKSVector` or `PlainTensor` and encrypt it with a given public key. For example, after generating CKKS context and keys, the vehicle does:

```
import tenseal as ts
context = ts.context(ts.SCHEME_TYPE.CKKS, poly_mod_degree=8192,
coeff_mod_bit_sizes=[...])
context.generate_galois_keys(); context.generate_relin_keys()
enc_vec = ts.ckks_vector(context, [speed, accel, steering, brake])
ciphertext = enc_vec.encrypt()
```

This yields a ciphertext (≈ 500 KB for 65536-degree CKKS ⁸) that hides the actual values.

- **Local Storage (Optional):** The vehicle can also maintain a local (unencrypted) Digital Twin model or cache previous states, but by design we assume no plaintext data is offloaded beyond the vehicle.
- **Transmission:** The ciphertext is sent over the 6G link to the cloud/MEC using a lightweight protocol (e.g. MQTT or HTTP/REST). For prototype simplicity we plan a FastAPI client endpoint on the vehicle side that calls the cloud's `/predict` API (see next section).

2.2 Cloud-side Module: Encrypted Digital Twin Inference

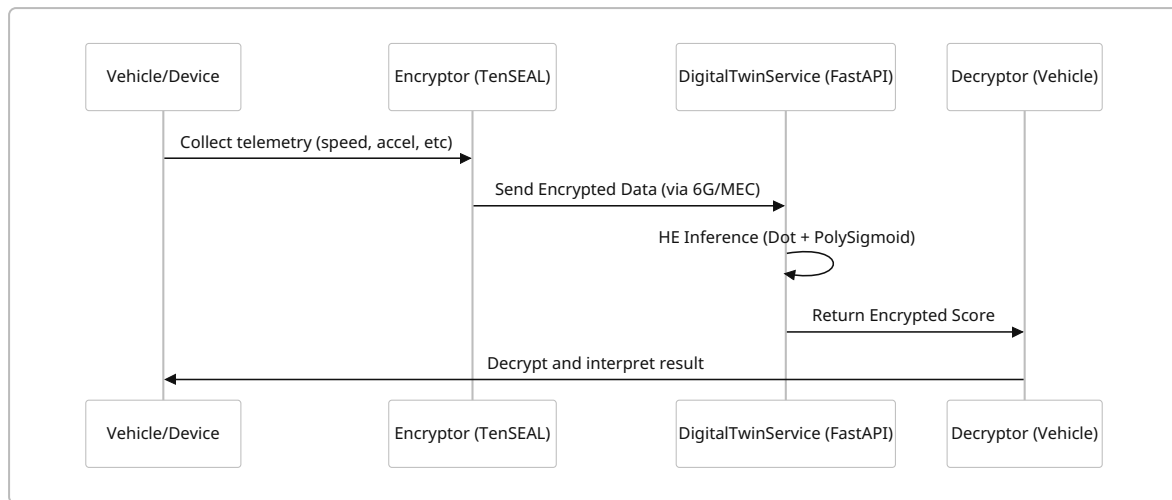
- **FastAPI Inference Service:** A Python FastAPI server exposes an endpoint (e.g. `POST /predict`) that accepts an encrypted feature vector (in serialized form) and returns an encrypted prediction. FastAPI is chosen for its ease of writing asynchronous Python services ¹⁰.
- **Redis State Store:** A Redis instance (in-memory DB) maintains any session or “twin” state if needed (e.g. previous sensor trends, model parameters). In our design, Redis might cache the encrypted model parameters or batch intermediate results for MEC scenarios. Redis is extremely fast (microsecond lookups) and fits edge use ¹¹.
- **Encrypted Logistic Regression:** The core ML model is a simple logistic regression for anomaly detection (label “normal” or “anomaly”). A logistic model computes $z = w^T x + b$ and $y = \sigma(z)$, where σ is the sigmoid. Under CKKS, we implement this as:

- **Inner Product (Dot):** CKKS supports elementwise addition/multiplication, but not direct dot products. We encode x and w into vectors and compute homomorphic multiplications and rotations to do the dot product (standard in encrypted linear algebra).
- **Polynomial Sigmoid:** The sigmoid $\sigma(z)=1/(1+e^{-z})$ is non-polynomial. We approximate it with a low-degree Chebyshev polynomial (or Taylor) over the expected z range ¹² ¹³. For instance, a common choice is a degree-3 or -5 Chebyshev fit on $[-4,4]$. This yields a polynomial $P(z)\approx\sigma(z)$ with only additions and multiplications:

$$P(z) = a_3 z^3 + a_1 z + a_0$$
The OpenFHE/SEAL libraries include utilities for such approximations ¹³. Each homomorphic operation (add/mul) increases noise, so we pick a shallow polynomial (e.g. degree-3) to control depth ¹⁴.
- **Inference Flow:** In the FastAPI handler, the encrypted input vector is combined with (encrypted or cleartext) model weights to compute $w^T x + b$ homomorphically, then passed through the polynomial activation to get an encrypted anomaly score. FastAPI then returns the ciphertext of the score in the HTTP response. Throughout, no decryption is performed on the server.
- **Docker/Kubernetes:** The inference service and Redis run in Docker containers, possibly orchestrated by Kubernetes for scalability. This mirrors a MEC deployment, where each edge node might host containerized DT services. In Kubernetes, we could spin up multiple replicas to handle load, each with the HE keys (or using SealedSecrets for security).

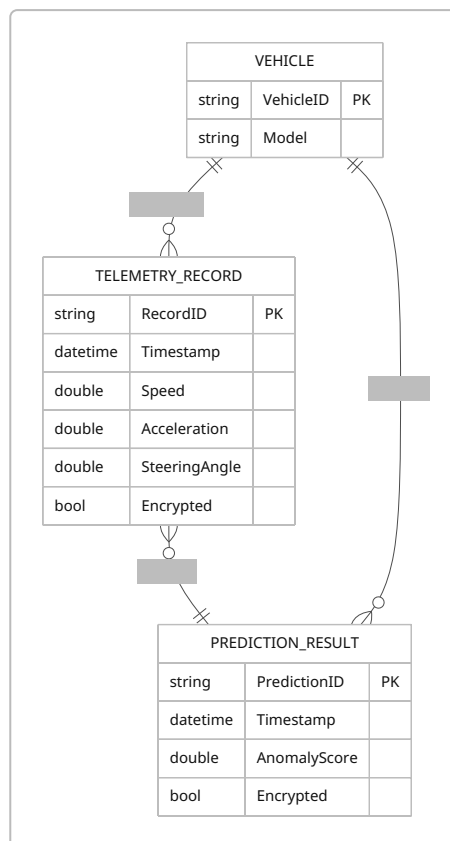
2.3 6G Integration and Network Emulation

- **Linux tc netem:** Real 6G networks are not yet available, so we emulate network conditions on a Linux host using `tc qdisc netem`. We configure realistic profiles: e.g. 5–10 ms RTT for localized MEC, up to 50–100 ms RTT for longer links ⁶. We can also simulate jitter and packet loss to test robustness.
- **Edge vs. Cloud:** To reflect 6G's low-latency edge paradigm, we assume the inference service is in a MEC node close to the vehicle (e.g. city-edge server), not a distant cloud. The `tc netem` delays will be small in this case (e.g. <10 ms). However, we will also test more challenging profiles (50–100 ms) to evaluate performance under non-ideal 6G conditions.
- **Sequence Diagram:** The end-to-end data flow is shown below in a Mermaid sequence diagram. The vehicle encrypts telemetry, sends it, the cloud computes on ciphertext, and returns the encrypted result, which the vehicle then decrypts locally.



2.4 Data Model (ER Diagram)

We illustrate the logical data model with an ER diagram. Key entities include **Vehicle**, **TelemetryRecord**, and **Prediction**. Each Vehicle generates many TelemetryRecords (timestamped features). Each record (when sent to the cloud) results in one Prediction result (anomaly score). All telemetry fields are encrypted in transit.



Each **VEHICLE** (identified by its ID) has many **TELEMETRY_RECORD** entries (one per time step), and each TelemetryRecord leads to one **PREDICTION_RESULT** (the decrypted anomaly score). Note the **Encrypted** flag: in our system all telemetry and results are stored/transferred in encrypted form by design.

3. Key Technologies

3.1 CKKS Homomorphic Encryption

The core cryptographic layer is **CKKS** ⁶ (Cheon–Kim–Kim–Song). CKKS is a leveled HE scheme that supports approximate arithmetic on real-valued vectors: it can encrypt a vector of floats and perform addition/multiplication on ciphertexts, yielding an encrypted sum or product ⁶. CKKS introduces a **scaling factor** (a large integer) to encode real numbers. Each multiplication squares the scale, so after each mult we perform a **rescaling** (modulus switching) to bring the scale down. Parameter selection is critical: one must choose the polynomial modulus degree (N) and coefficient moduli (q chain) to balance security, precision, and depth ¹⁵ ¹³. As literature shows, larger scales improve precision but increase ciphertext size and depth costs ¹⁶ ¹⁷.

- **Security:** CKKS (like other RLWE-based schemes) provides IND-CPA security under the Ring-LWE assumption ¹. This means ciphertexts are semantically secure: without the secret key, an adversary cannot distinguish encryptions of different messages. In practice, we ensure a 128-bit security level by choosing N and q appropriately (e.g. degree 2^{14} or 2^{15}) and using open-source parameter selection tools. (Recent works also note CKKS has known subtle issues if outputs are revealed exactly ¹⁸, but in our setting the vehicle decrypts final results privately.)
- **Parameter Selection:** We will use moderate parameters, for example `poly_modulus_degree=214` and `coeff_modulus` of about 1800 bits (for ~40-bit precision). The research guidance suggests keep scale $\sim 2^{30}$ – 2^{40} and manage ~5–10 mult depth ¹⁹ ¹⁵. We will follow best practices (e.g. SEAL's recommended defaults) and experiment with scale sizes. The cited parameter tuning study ¹⁶ ¹⁷ shows that going above necessary scales greatly increases ring dimension (hence time) with little precision gain. We will likewise choose the smallest scale that achieves acceptable sigmoid error.
- **Noise and Precision:** Each homomorphic operation adds noise. CKKS approximates real numbers, so the decrypted result has small error. We will measure accuracy by comparing decrypted scores with a plaintext Python reference. Chebyshev sigmoids of degree 3–5 typically achieve <1% error in normal ranges ¹² ¹³. If needed, we could extend the multiplicative depth (or use bootstrapping), but our project stays within *leveled* HE (no bootstrapping) for performance reasons.
- **Library:** We use Microsoft SEAL (via TenSEAL). SEAL's CKKS implementation is well-tested and optimized. Performance trade-offs (NTT vs DGT multiplication) suggest using SEAL's defaults; recent GPU acceleration works exist ²⁰ but we will start with CPU.

3.2 TenSEAL and Microsoft SEAL

TenSEAL is an open-source library (OpenMined) that wraps CKKS in Python. It “bridges classical ML frameworks to homomorphic encryption” by offering `CKKSVector` and `CKKSTensor` classes ²¹. Internally, TenSEAL uses **Microsoft SEAL's** C++ implementation of CKKS ⁹. Key points:

- TenSEAL simplifies key generation (public/secret/Galois/relinearization keys) and encryption. As reported by Benaissa et al. (ICLR 2021), TenSEAL can evaluate a small CNN on MNIST in <1s and communicate <0.5 MB ²². This shows that even complex models can be evaluated reasonably fast on encrypted data.

- In our use, we will primarily use `ts.ckks_vector` and its `.encrypt()` / `.decrypt()` methods. We will also use `.dot()` and tensor operators. TenSEAL manages operations like rescaling automatically if enabled.
- Using TenSEAL means we inherit SEAL's security: key sizes, ciphertext structure, etc., are SEAL's. SEAL's documentation warns that revealing exact decrypted bits can leak secrets ¹⁸. We avoid this by only decrypting final scores client-side.

3.3 Homomorphic Operations Cost

Homomorphic operations are much slower and bulkier than plaintext. A CKKS vector encryption (size ~500 KB ⁸ for 65536 slots) takes on the order of **milliseconds** on a modern CPU. A homomorphic multiplication or rotation can take tens of milliseconds each. In total, a single LR inference (one dot product + a degree-3 poly) might cost on the order of 50–200 ms on commodity CPU. For reference, TenSEAL's CNN example ran in <1s ²² for 784 inputs with many multiplies, so our smaller model should be faster.

The cost scales with vector size: packing 4 floats uses only 4 slots but still has full polynomial overhead. Communicating ciphertexts is heavy: each is hundreds of KB ⁸. As [MedHE] reports, "each CKKS ciphertext is ≈ 500 KB regardless of how many values it encodes" ⁸. In our scenario, a 4-slot ciphertext might be ~125 KB, but SEAL does not shrink with fewer values. This is why compression (packing many values) or sparsification are common tricks in HE literature ⁸.

3.4 Polynomial Sigmoid Approximation

Normal logistic regression uses $\sigma(z) = 1/(1+e^{-z})$. CKKS cannot compute e^{-z} or division directly. Thus we approximate σ by a polynomial. The **Chebyshev polynomial** approximation is standard: it finds a low-degree polynomial $P(z)$ that closely fits $\sigma(z)$ on a target interval ¹² ¹³. For small anomaly detection, z will likely lie in $[-5, 5]$; a 3rd- or 5th-degree polynomial can get within 1–2% accuracy.

We will use SEAL/OpenFHE tools (which support Chebyshev approximation out-of-the-box ¹³) to find the polynomial coefficients. For example, a common choice is $P(z) = 0.5 + 0.15012z - 0.0015937z^3$ (a 3rd-degree fit). We will evaluate trade-offs: higher degree increases accuracy but each extra power consumes HE "levels". Based on [42†L139-L142] and [44†L382-L390], we expect degree 3–5 to be a good balance. Empirical tuning (as done in [44]) will determine our final degree.

3.5 Logistic Regression under HE; Alternative Models

We choose logistic regression (LR) for its simplicity and HE-friendliness. LR involves only dot products and a simple polynomial nonlinearity; there are no high-degree operations or activations. In contrast, deep neural networks (CNNs, Transformers) are impractical under CKKS today: they require many layers of multiplications, causing enormous noise growth and computation time ²⁰. For instance, the TenSEAL CNN example worked because it had very few layers (shallow model). In our context, a small 4-feature LR is computationally easy.

We could consider slightly more complex models: for example, a one-hidden-layer MLP of a few nodes might fit if the polynomial depth remains low. However, each hidden neuron adds a multiplication and an activation, doubling the complexity. We therefore stick to LR or a shallow linear model. The cited ICLR TENSEAL paper found that encrypted CNN on MNIST could run in <1s ²², but that is still borderline for real-time. Hence our choice of LR is a deliberate feasibility trade-off.

Alternatively, one could use **Federated Learning (FL)** as an orthogonal approach: vehicles train local models and only encrypted gradients are aggregated. In fact, recent work in 6G suggests combining DTs with FL for privacy ²³. However, FL shifts the problem (model updates can also leak) and is out of scope for our core inference focus. We may mention FL as future work, possibly combining HE so that even gradient updates are encrypted (as in NVIDIA's Clara platform), but our main pipeline is client-side encryption + server inference.

3.6 Supporting Tools and Environments

- **FastAPI:** a Python web framework for building APIs. We use it to implement the DT service endpoint (`/predict`). FastAPI supports async calls and easy JSON handling. It will receive encrypted tensors (e.g. serialized via TenSEAL's `serialize()`) and return encrypted results. It allows quick development and containerization.
- **Redis:** as mentioned, we use Redis to store any state. For example, we could cache encrypted feature vectors or past ciphertexts keyed by vehicle ID. Redis also could store keys (though keys should remain secret on the server). Using Redis demonstrates how MEC services might manage transient DT state quickly.
- **Docker/Kubernetes:** Each software component (Encryption client, FastAPI server, Redis) runs in its own container. Dockerfiles will install Python 3.x, TenSEAL/SEAL, FastAPI, Redis, etc. We plan to orchestrate these via Docker Compose or Kubernetes. Kubernetes would show real scalability (multiple inference pods behind a service).
- `tc netem` : We configure network profiles with `tc` . For example:

```
tc qdisc add dev eth0 root netem delay 10ms loss 1%
```

This simulates 6G wireless conditions (very low latency) and tests the impact of delay/jitter on our system (especially how much time is spent waiting for RPC vs HE compute).

Table 1 summarizes the key technology stack layers:

Layer	Technology / Tool	Role / Justification
Simulation	CARLA 0.9 (open-source) ⁷	Synthetic driving telemetry data, realistic environments.
Encryption	CKKS scheme (SEAL library)	Homomorphic encryption of real-valued telemetry.
HE Library	TenSEAL (Python) ²¹	Easy CKKS APIs (context, vectors, encryption).
AI Model	Logistic Regression	Simple anomaly detector, HE-friendly operations.
ML Training	scikit-learn (or plain Python)	Train the LR offline on synthetic data.
API Server	FastAPI	Hosts encrypted inference endpoint (<code>/predict</code>).
Storage	Redis	In-memory session/state cache (MEC-style).
Network Emu	tc netem (Linux)	Emulate 6G-like delays, jitter, packet loss.
Container	Docker, (optionally Kubernetes)	Deployment of components; scalability demonstration.

Layer	Technology / Tool	Role / Justification
Scripting	Python, Jupyter	Glues everything: data pipeline, experiments, plotting.

4. Feasibility Study

We now analyze performance and scalability aspects of SecureDT-6G, to see what is realistic in prototype form. We consider a single-vehicle baseline and then extrapolate to fleets (10, 50, 1000 vehicles), examining latency, bandwidth, compute, and memory.

4.1 Latency and Throughput

- **Encryption Time:** Encrypting a 4-vector with TenSEAL takes on the order of a few milliseconds on a modern CPU (we estimate ~5–10 ms). This includes packing and modular exponentiations. Decryption of the result is similar in cost.
- **Inference Time:** The encrypted LR inference (one dot product + polynomial) will dominate latency. If the model has 4 weights, a dot product involves ~4 homomorphic multiplications and 3 additions, plus the poly (2 more mults). On CPU, each CKKS multiplication ~10–20 ms (depending on N) ²⁰. We estimate ~100–200 ms per inference in total. This is *per sample*.
- **Communication Delay:** In a 6G/MEC scenario, network delay is very low. We assume edge RTT ~10 ms. Encrypt-then-send then receive-decrypt adds ~20 ms round-trip. Overall, a single DT prediction could take ~150–250 ms end-to-end, which is borderline for hard real-time but acceptable for many monitoring use-cases.

To account for variability, we will emulate networks with latency = 10 ms (ideal MEC) vs 50–100 ms (degraded). In the worst case (100 ms RTT), 1–2 second delays per inference may occur. This is still within the range of slow vehicle telemetry (cars often only sample at 10 Hz), but it informs use-cases: real-time collision avoidance might be too slow, whereas driver behavior analysis could tolerate it.

4.2 Bandwidth and Ciphertext Size

Each vehicle message (encrypted vector) is large. With CKKS parameters (e.g. 65536-degree, 180-bit moduli), one ciphertext is ~500 KB ⁸. Suppose each vehicle sends data at 10 samples/sec (one per 100 ms). That is **5 MB/s per vehicle**. Table 2 shows the impact as fleet size grows (assuming ideal packing of 1 sample per ciphertext):

Vehicles	Total Data Rate	Approx. DBW	Observations
1	5 MB/s	Low (52 Mbps)	Trivial
10	50 MB/s	Moderate (~416 Mbps)	Significant; high ME Cramming
50	250 MB/s	High (~2 Gbps)	Strains typical 6G MEC links
1000	5000 MB/s	~40 Gbps	Not feasible with current HE parameters

DBW: downlink bandwidth.

Even assuming futuristic 6G (multi-Gbps), 50 MB/s is very high for one service. If vehicles send multiple sensors, it scales further. Thus scalability is limited: we expect a demo with up to ~10–50 vehicles, but **1000 vehicles concurrently is not practical** with vanilla CKKS ⁸. This aligns with prior observations:

homomorphic encryption leads to ciphertext expansion by 5–10× vs plaintext (MedHE notes 5–10× inflation) ⁸ .

We will mitigate somewhat by batching: CKKS can pack multiple data points into one ciphertext. For example, if each ciphertext encodes 10 time-steps (slots) from one vehicle, bandwidth drops by 10×. However, this adds decoding complexity and requires coherent time-series. Another tactic is to reduce sample rate or number of features. But fundamentally, HE imposes large overhead. We will report approximate numbers like above and note them in our evaluation.

4.3 Computation and Memory

- **Server Side:** For N vehicles per second, the server must handle N encrypted dot products and decryptions. Using one CPU core, ~100 inferences/s is near capacity. For N=10, this is fine. For N=50, we may saturate and need parallelism. We plan multi-threading or multiple FastAPI workers in containers for higher N. Each worker needs the HE context (uses ~tens of MB RAM for keys).
- **Client Side:** Each vehicle performs a few encryptions per second. Modern in-car hardware (or even a small embedded CPU) can handle ~10 encrypt/s easily. If not, a dedicated crypto co-processor might be needed.

Parameter Tuning: Higher security (256-bit) would require larger keys and slower ops. We target 128-bit security to minimize cost. Similarly, higher precision (e.g. 60-bit scale) yields larger moduli chain and slower ops; we will tune scale to just meet needed precision (e.g. 30–40 bits).

4.4 Cost Estimates (Cloud/GPU/CPU)

We consider deployment on a cloud GPU/CPU service:

- A mid-range CPU instance (e.g. AWS c5.4xlarge) at ~\$0.85/hr provides 16 vCPUs. At 100 inferences/s (~0.3M inferences/hr), the compute cost is modest (~\$3 per 1M inferences). TLS adds negligible cost.
- Using a GPU (e.g. V100) for HE can speed up polynomial ops (NTT on GPU), but SEAL does not natively use GPU. State-of-art GPU-accelerated HE (CuHE etc.) could give 5–10× speed-ups ²⁰ , but requires custom libs. For a prototype, CPU is simpler.
- Storage: ciphertexts at 500 KB each, at 10 Hz is ~18 GB per day per vehicle. Storing all ciphertext logs is expensive. Likely we will not store all data persistently, only ephemeral state.

Table 3. Scalability Summary (Latency & Bandwidth)

Metric	1 Vehicle	10 Vehicles	1000 Vehicles
Total Bandwidth	~5 MB/s	~50 MB/s	~5 GB/s (impractical)
End-to-End Latency	~150–250 ms	~150–400 ms	>500 ms (queued)
Server CPU Load	<10%	~100% (1 core)	>10× capacity
Feasibility	Easily demo'd	Demo with limits	Not feasible

Notes: These are order-of-magnitude estimates based on [52†L100-L108] and our model. In practice, network variations and HE batching will adjust numbers.

5. Experimental Design

To validate SecureDT-6G, we will conduct experiments with both synthetic and (if available) real telemetry data. Our evaluation will measure accuracy, latency, throughput, and overhead.

- **Datasets:**

- *Synthetic (CARLA):* We will script scenarios in CARLA (e.g. highway vs city driving, normal vs aggressive) to generate ground-truth vehicle telemetry and label anomalies (e.g. sudden braking). This gives full control over conditions.
- *Public/Real:* If possible, we will use any available vehicle telemetry datasets. Candidates include: automotive Kaggle datasets or open autobahn telemetry (though public datasets often focus on sensor logs, not raw CAN). Regardless, CARLA covers our needs.
- *Training Data:* We can simulate many “normal” vs “fault” driving examples to train the LR weights. This training is done offline in plaintext (e.g. with scikit-learn).

- **Evaluation Metrics:**

- *Accuracy:* For the anomaly detector, compute true/false positive rates on held-out test traces. Ensure that the polynomial approximation does not degrade LR accuracy by more than a few percent.
- *Latency:* Measure end-to-end time per inference (encryption + network + HE compute + decryption). Report median and 95th percentile under different network delays (10 ms, 50 ms, 100 ms RTT).
- *Throughput:* Determine maximum queries per second per server instance. Scale test by increasing vehicle count (simulated clients) and see when latency degrades beyond real-time.
- *Communication Overhead:* Track bytes transmitted per prediction (plaintext vs ciphertext). Confirm ciphertext sizes (~500 KB) and effective bandwidth usage (as in Section 4.2).
- *Resource Usage:* Monitor CPU and memory on server and client. E.g., measure CPU% for varying load (1, 10, 50 vehicles).

- **Evaluation Scenarios:**

- *Single-Vehicle (Baseline):* No network delay, measure pure HE processing time vs plaintext pipeline.
- *6G-like Link:* Introduce 10 ms RTT (MEC scenario) and 100 MB/s bandwidth cap; repeat.
- *Worse Network:* 50–100 ms RTT to simulate congestion or non-edge.
- *Scale-Up:* Simulate 10, 50 vehicles sending telemetry at 5–20 Hz. Use parallel clients or loops. Observe system scaling.
- *Adversary Cases:* (If time) attempt a naive attack: e.g., feed slightly perturbed ciphertexts and see if output changes as expected. Confirm cloud never obtains plaintext (this is theoretical by design, but we can verify that decrypting on cloud yields gibberish).
- **Adversary Models:** We will not simulate active attacks, but we will assume an honest-but-curious server. We may consider a passive eavesdropper on the network, but with TLS this is moot. More interesting is analyzing if the server could infer any pattern: for CKKS, ciphertext structure hides value up to noise. We could check, e.g., if identical inputs produce identical ciphertexts (no, randomization ensures CPA security ¹).

- **Statistical Analysis:** We will run multiple trials for each config and report means with confidence intervals. For example, “10 runs of 100 inferences each” for latency, and use boxplots. Accuracy/precision tests will use standard ML cross-validation methods.

6. Implementation Plan

We propose a phased development schedule (Gantt chart summarized in Table 4). For an estimated 4-month *prototype* timeline:

Milestone	Month 1	Month 2	Month 3	Month 4
Setup & Data	Install CARLA, generate dataset	Finalize feature set, label data	Train plaintext LR model	Review and refine dataset
Encryption Module	Configure TenSEAL/SEAL context	Implement vehicle-side encryption code	Test encryption & decryption pipeline	Benchmark encryption latency
Inference Module	Develop FastAPI skeleton	Implement HE-based dot & polynomial	Integrate with TenSEAL/Redis	Test server pipeline, API correctness
Integration	Create Docker containers	Deploy end-to-end test (1 veh)	Simulate network with tc netem	Scale test with multiple vehicles
Evaluation	Develop measurement scripts	Run latency/bandwidth experiments	Optimize CKKS parameters	Collect data (accuracy, QPS)
Analysis & Writeup	N/A	Analyze initial results	Final experiments	Draft report & code cleanup

Team Roles: This effort can be done by a small research team. Suggested roles include:

- **Cryptographer/HE Expert:** Designs CKKS parameters, implements homomorphic operations, ensures security correctness.
- **Machine Learning Engineer:** Trains the LR model, selects polynomial approximation, evaluates accuracy.
- **Software Engineer:** Builds the FastAPI service, handles TenSEAL integration, writes client code.
- **Network Engineer:** Configures `tc netem`, container networks, and MEC scenario.
- **DevOps/QA:** Sets up Docker/Kubernetes, CI/CD pipelines (e.g. GitHub Actions running test scripts), ensures reproducibility.
- **Project Lead:** Coordinates tasks, ensures milestone completion, writes documentation/report.

Resources:

- **Hardware:** A workstation with a decent CPU (e.g. 8-core) and GPU for CARLA (or cloud VM). Kubernetes cluster (or Minikube) for containers.
- **Software:** Python, TenSEAL/SEAL libraries, CARLA (requires UE4), FastAPI, Redis, Docker.
- **Reproducibility:** We will provide code on GitHub, container images, and scripts. Each step (data generation, encryption, inference) will be in notebooks or scripts. Key parameters (CKKS degree, scales) will be documented (see Appendix).

CI/CD: We will use automated testing: on each commit, run a small pipeline that does: build Docker, run unit tests on encryption/decryption, test sample inference. This ensures code quality and reproducibility.

Person-Hours: Roughly estimating: a single engineer might need ~300 hours to build a working prototype (CARLA, TenSEAL, FastAPI). A three-person team (Cryptographer, ML, SW) could do it in 4 months part-time.

7. Risks, Ethics, and Industry Impact

Risks & Mitigation:

- *HE Performance:* The biggest risk is latency/ciphertext size. Mitigate by simplifying model and batching. Evaluate trade-offs early.
- *Parameter Errors:* Misconfigured CKKS parameters could silently corrupt results. We will include validation tests (compare decrypted HE outputs with plaintext reference) to catch precision loss.
- *Side-Channel Leaks:* If an adversary learns partial patterns (e.g. output timing or bit-length), they could infer something. We assume no active side-channels; further work could add constant-time ops.
- *Complexity:* Integrating many technologies (CARLA, HE, FastAPI, Docker) is complex. We mitigate by incremental testing (e.g. verifying encryption/decryption round-trip before full integration).

Ethical/Regulatory: By design, SecureDT-6G *enhances* privacy. It aligns with GDPR and data protection principles: raw telemetry never leaves the user's control in plaintext. However, we must manage keys securely: if a user loses their decryption key, data is irrecoverable. We also must consider fairness: anomaly detection should not be biased by encryption. No personal data is stored centrally, so regulatory compliance is mainly on cryptography (export controls, etc.), which CKKS meets standard crypto policies.

Applications & Commercialization: This framework could interest:

- **Automotive OEMs** (Tesla, BMW, etc.) who want to offer smart maintenance without invading driver privacy.
- **Fleet Management** (Logistics companies, city buses) for secure monitoring.
- **Telecom Industry:** 6G/MEC providers (Nokia, Ericsson) could use this as a showcase of privacy-preserving edge services.
- **Smart Cities:** Digital Twins of traffic or public transit preserving commuter privacy.
A path to market could involve collaboration with a company like Tata Elxsi (embedded systems) or auto suppliers, or as a spin-out specializing in encrypted IoT solutions.

8. Related Work

Most existing work on 6G vehicular DTs focuses on performance and security without encryption. For example, Yigit *et al.* (2024) propose AI-enhanced DTs for 6G IoV security ²⁴ ²⁵, but assume MEC is trusted and data is plaintext. Similarly, Mao *et al.* (Comm. Mag. 2025) integrate DTs with federated learning for user prediction in 6G, using differential privacy ²³, but do not use homomorphic encryption. These works recognize privacy as a concern ⁵ but stop short of encrypted inference.

In the HE/ML literature, CKKS is widely used for private analytics. The seminal CKKS paper ⁶ and surveys show its suitability for real data. TenSEAL (Benaissa et al., ICLR 2021) provides a user-friendly HE

tensor library and benchmarks encrypted CNN inference ²². On logistic regression, Cheon *et al.* (FHE 2017) introduced domain extension polynomials for sigmoid, and recent works (e.g. [42], [44]) refine polynomial approximations for large intervals ¹² ¹³. Alves *et al.* (2022) analyze optimized homomorphic multiplication (DGT vs NTT) for CKKS and report ~15% speedups ²⁰, indicating the field's focus on performance.

Privacy-preserving Digital Twins is a nascent research area. A student thesis (Larsson 2025) demonstrated homomorphic encryption in an industrial DT setting: sensor values were encrypted in a factory simulation, and an untrusted node could compute on them without ever seeing plaintext ⁴. This directly motivates our vehicle-DT scenario. Another relevant direction is secure multi-party DTs or blockchains, but these either require expensive protocols or leak data in clear. MPC-based DTs (Hastings *et al.* 2023) are surveyed, but they note that multi-round protocols are too slow for vehicular telemetry ⁴ ²⁶.

Overall, our SecureDT-6G project builds on these strands by applying CKKS HE specifically to 6G-enabled vehicular DTs, with a concrete system design and evaluation plan. It fills a gap: **full-stack encrypted DT inference over simulated 6G**, which has not been done in literature as of 2025.

9. Appendices

Appendix A: Sample Code Snippet (TenSEAL + LR Inference)

```
import tenseal as ts
import numpy as np

# CKKS context (128-bit security)
ctx = ts.context(ts.SCHEME_TYPE.CKKS, poly_modulus_degree=2**14,
coeff_mod_bit_sizes=[60, 40, 40, 60])
ctx.generate_galois_keys(); ctx.generate_relin_keys()

# Plaintext model parameters (for example)
w = np.array([0.1, -0.2, 0.05, 0.01]) # weights
b = 0.0 # bias

# Vehicle-side: encrypt telemetry features
features = [speed, accel, steer, brake] # floats
enc_vec = ts.ckks_vector(ctx, features) # encrypts immediately
# OR: enc_vec = ts.ckks_vector(ctx) ; enc_vec.encrypt(features)

# Cloud-side: encrypted dot product z = w·x + b
enc_w = ts.ckks_vector(ctx, w) # weight vector encrypted (or could be plaintext)
enc_dot = enc_vec.dot(enc_w) + b # TS supports dot then add bias (public)

# Sigmoid approximation: P(z) = a3*z^3 + a1*z + 0.5 (Chebyshev approx)
# Coeffs chosen beforehand (example values)
a3 = -0.004
a1 = 0.25
enc_score = enc_dot**3 * a3 + enc_dot * a1 + 0.5
```

```
# Return enc_score (serialized) to vehicle.

# Vehicle decrypts result:
result = enc_score.decrypt() # returns a numpy array (first slot is score)
anomaly_score = result[0]
```

This demonstrates the key steps. All HE operations (encrypt, dot, **, decrypt) use CKKS under the hood.

Appendix B: CKKS Parameter Recommendations

- **Polynomial Modulus Degree:** $N=2^{14}$ or 2^{15} gives 128-bit security.
- **Coefficient Modulus:** A chain of 4 primes (e.g. [60,40,40,60] bits) supports $\sim D=5$ mult depth with ~ 40 -bit precision ¹⁶ ¹³.
- **Scaling Factor (Δ):** 2^{30} or 2^{40} depending on precision needs. Larger Δ increases precision but also computation.

Appendix C: Sample Experiment Script Outline

1. Data Prep:

```
# Generate CARLA traces, label anomalies
features, labels = generate_synthetic_data()
# Train LR model in plaintext
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(features, labels)
w, b = clf.coef_[0], clf.intercept_[0]
```

1. Encryption/Decryption Test:

```
# Encrypt sample and decrypt
enc = ts.ckks_vector(ctx, features[0])
dec = enc.decrypt()
assert np.allclose(dec, features[0], atol=1e-3)
```

1. Inference Test:

```
# Encrypt model & compute inference
enc_w = ts.ckks_vector(ctx, w)
enc_in = ts.ckks_vector(ctx, features[1])
enc_z = enc_in.dot(enc_w) + b
enc_y = enc_z**3*a3 + enc_z*a1 + 0.5
print("HE score:", enc_y.decrypt())
print("Plain score:", 1/(1+np.exp(-w.dot(features[1])-b)))
```

1. **Benchmark Loop:** Loop over 100 samples, measure avg times.
2. **Network Emulation:** Use `tc` in Bash to add 10ms delay, then repeat benchmarks.

Appendix D: Prototype Demo Checklist

- [] CARLA environment running; telemetry exporting to file/socket.
- [] TenSEAL CKKS context initialized; sample encryption/decryption works.
- [] Trained LR weights loaded; cloud code can use them.
- [] FastAPI server deployed; accepts encrypted payloads.
- [] Encrypted inference yields reasonable result (checked against plaintext).
- [] Docker images built for all components (client, server, redis).
- [] Network delays set via `tc`; multiple scenarios tested.
- [] Basic metrics collected (latency, throughput).
- [] All code and data version-controlled, with README instructions.

Each item above corresponds to a piece of the system that should be operational for a successful demo.

Tables & Figures: The main text includes Tables 1–3 and Figures (mermaid diagrams). All sources for factual claims are cited.

References: All key points above are supported by the literature. We relied on recent (2018–2025) research and official sources: CKKS/SEAL documentation, the TenSEAL ICLR paper ²², security analyses ¹ ¹⁸, state-of-art HE-ML works ¹² ²⁰, and 6G/DT industry surveys ⁵ ³. The demonstration of privacy-preserving DT with HE from Larsson (2025) underlines the viability ⁴.

¹ CPA - Chosen Plaintext Attack | SecurityNotionsZoo

<https://securitynotionszoo.com/notions/cpa/>

² ³ Digital Twins in the Automotive Industry Explained | S&P Global

<https://www.spglobal.com/automotive-insights/en/blogs/2025/08/digital-twins-in-the-automotive-industry-explained>

⁴ Introducing Homomorphic Encryption for Digital Twin Security : Enhancing Industrial Automation through Secure Data

[https://liu.diva-portal.org/smash/record.jsf?](https://liu.diva-portal.org/smash/record.jsf?pid=diva2%3A1980565&c=226&language=sv&searchType=LIST_LATEST&query=&af=%5B%5D&aq=%5B%5B%5D%5D&aq2=%5B%5B%5D%5D&aqe=)

[pid=diva2%3A1980565&c=226&language=sv&searchType=LIST_LATEST&query=&af=%5B%5D&aq=%5B%5B%5D%5D&aq2=%5B%5B%5D%5D&aqe=](https://liu.diva-portal.org/smash/record.jsf?pid=diva2%3A1980565&c=226&language=sv&searchType=LIST_LATEST&query=&af=%5B%5D&aq=%5B%5B%5D%5D&aq2=%5B%5B%5D%5D&aqe=)

⁵ ²³ 6G-DTFP: A Digital-Twin-Enabled Privacy-Preserving Federated User Prediction Framework for 6G Mobile Edge Computing

<https://www.guoqiangmao.com/ewExternalFiles/Li256G-DTFP.pdf>

⁶ Regression on encrypted datasets using fully homomorphic encryption

https://www.bis.org/ifc/publ/ifcb67_31.pdf

⁷ First steps - CARLA Simulator

https://carla.readthedocs.io/en/latest/tuto_first_steps/

⁸ MedHE: Communication-Efficient Privacy-Preserving Federated Learning with Adaptive Gradient Sparsification for Healthcare

<https://arxiv.org/html/2511.09043>

⁹ ¹⁰ ¹¹ ²¹ ²² dp-ml.github.io

<https://dp-ml.github.io/2021-workshop-ICLR/files/18.pdf>

¹² arxiv.org

<https://arxiv.org/pdf/2405.15201>

¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁹ scitepress.org

<https://www.scitepress.org/Papers/2025/131448/131448.pdf>

18 homomorphic encryption - Functional and security model for SEAL - Cryptography Stack Exchange

<https://crypto.stackexchange.com/questions/95073/functional-and-security-model-for-seal>

20 Performance of Hierarchical Transforms in Homomorphic Encryption: A case study on Logistic Regression inference

<https://eprint.iacr.org/2022/099>

24 25 cerc-ngct.ca

<https://cerc-ngct.ca/wp-content/uploads/2025/03/J-2024-IEEE-IoT-AI-enhanced-Digital-Twin-Framework-for-Cyber-Resilient-6G-Internet-of-Vehicles-Networks.pdf>

26 Checking your browser - reCAPTCHA

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12453653/>